Visualization of LLL and BKZ Algorithms with Vector Representations

Muhammad Imam Chaidar Mujaddid Al-Ghiffari and Amril Syalim

Faculty of Computer Science, Universitas Indonesia

Abstract. As the quantum computing starts to threaten the current cryptographic systems such as RSA, cryptographers around the world start to develop the Post-Quantum Cryptography (PQC) algorithms, most of them are lattice-based (for example ML-KEM and ML-DSA), which are strong enough to withstand attacks from quantum-based cryptanalysis techniques such as Shor's algorithm, Grover's algorithm, and other quantum cryptanalysis algorithms. In order to strengthen these PQC algorithms, cryptographers and cryptanalysts alike also developing LLL and BKZ algorithms to test the strength of these PQC algorithm. One of the problems with LLL and BKZ is the fact that while these algorithms are able to penetrate PQC systems, they're harder to learn compared to the algorithms which they succedeed. Sufficient mathematical understanding is required in order to get proper understanding of these algorithms. Some online tools have been available, especially for LLL algorithm, in order to visualize these algorithms so learners can learn them easier. The problem is that many of these visualizations only show the initial and the final result of the input. In this paper, we will build a tool to visualize the processes of both LLL and BKZ algorithms. Through visualization with vectors on Cartesian coordinate as well some panels such as Lovasz status panel and other panels that will explain the current stage of the process of both LLL and BKZ, we expect cryptography learners and enthusiasts alike to understand LLL and BKZ easier.

Keywords: Cryptography · Cryptanalysis · Post-Quantum · Lattice · LLL · BKZ.

1 Introduction

Cryptanalysis is the technique of analysing and breaking a ciphertext to obtain the plaintext (A. Al-Sabaawi, 2020). The history and the development of cryptanalysis itself can't be separated from the history and development of cryptography. With the onset of the development of quantum computing and subsequently quantum cryptanalysis, many commonly used modern cryptographic algorithms, especially RSA, are threatened by quantum cryptanalysis (). Which is why cryptographers around the world already started to develop cryptographic algorithms that can resist quantum cryptanalysis - the post-quantum cryptography (PQC) (). Most of PQC algorithms that are seen as promising, are lattice-based (Shaik

Ahmadunnisa et al, 2025). For example, Kyber and Dilithium and their standardized versions - the ML-KEM and ML-DSA algorithms. At the same time, cryptanalysts and cryptographers also use these lattice reduction cryptanalysis algorithms in order to evaluate the strength of these lattice-based PQC algorithms to resist their greatest foe - none other than the lattice reduction cryptanalysis algorithms ().

The most common lattice reduction technique is the LLL algorithm, which was already developed long before the advent of quantum computing, and obviously predates the PQC algorithms as well. A polynomial-time algorithm for finding a short and nearly orthogonal basis for a lattice, this algorithm can be used to solve the shortest vector problem (SVP) and the closest vector problem (CVP), which are essential in lattice-based cryptography. We could say that LLL is the foundation of the lattice reduction cryptanalysis algorithms, and to master the subsequent algorithms, mastering the LLL before is a crucial step.

For now, the most promising lattice-based cryptanalysis technique is the BKZ algorithm, which is basically the upscaled version of the LLL algorithm (). Compared to LLL, BKZ can find even shorter vectors in a lattice () and theoritically speaking, BKZ is a far greater threat to lattice-based PQC systems compared to LLL, which are no longer a major threat for these lattice-based PQC systems. To help learners learn LLL and BKZ algorithms, some online solver tools have been developed, for example fplll and fpylll, Sage Cell, and lll-attack-runner.

The problems with these LLL and BKZ solvers is that they are either only show the final result, less user-friendly compared to simpler matrix tools, doesn't have visualization, or doesn't show the mathematical processes between the initial state and the final result. The first problem is true for fplll, fpylll and lll-attack-runner, as they only show the final result. The second problem is true for Sage Cell, as we can't just insert a "plain" matrix to process it, we have to wrap the matrix within either a Python code, a Sage code, or any code in at least one of languages available in the tool. To put it simply, it isn't as practical as inserting the matrix in some simpler online matrix tools, such as Calculator.net, matrixcalc.org, or Desmos. And the third problem is true for fplll and fpylll as well Sage Cell, as they don't even have built-in visualization. Sage Cell can show every step of the LLL process, however, we need to modify the code we inserted in the tool, in a manual way, so we can't use the built-in method of .lll(), and we have to use our own code. For the fourth problem, all previously mentioned LLL and BKZ algorithms solvers have this problem. None of these solvers show the mathematical processes between the initial state and the final result. For Sage Cell, there is no built-in library to show these steps.

So in short, these LLL and BKZ tools aren't practical compared to several online math tools, and they don't automatically provide visualization from the beginning to the final result nor that they provide the step-by-step explanation. Which is actually an important issue, because psychologically wise, humans are generally easier to process information visually (Mayer, 2001) but also how the users interact with them is also an important factor (Christopher D. Hundhausen

et al, 2002). So we definitely need step-by-step explanations just like we need the visualizations, within a system that lets the user to interact with the system from the input insertion to the output visualization. For the visualization part, it's clear that visualization through vectors can significantly help users to understand the process. Why? Because the basis concept for LLL and BKZ is lattice reduction. Which is a linear algebra concept. And it is hard to teach the concepts of linear algebra without vectors (or, to be more precise, vector geometry) visualization (Konyaliouglu et al., 2011).

Our idea is that by visualizing LLL and BKZ algorithms from the beginning of the process until the final result with vectors on 2D and 3D Cartesian coordinates, showing the calculations behind the algorithms, as well some simple theoritical explanations of these algorithms, we can help the users to understand these algorithms easier. The inputs are in the form of matrices. We will develop a tool that not just process LLL and BKZ algorithms, but also show the visualization from the initial state until the final result, with simplified processes shown to help users understand these two algorithms easier. Our main idea for the visualization is through the usage of vectors within a Cartesian field, both in 2D and 3D projections, as well several panels that showing the current process (and its explanation), Lovasz condition status, and a button that can be pressed to go to the previous and next steps. We hope that through these visualization and textual elements, users would have it easier to understand the mechanism of both LLL and BKZ algorithms, which are increasingly becomes more relevant for the security of our data.

2 Related Works

2.1 Learning with Errors(LWE)

Learning with Errors (often shortened to LWE) is very essential for modern lattice-based PQC algorithms, such as NIST's ML-KEM and ML-DSA (Wang et al., 2025). Basically, this is a method to hide the information (e.g plaintext) by injecting randomized noises (errors) into a system of linear equations which represents the public matrix, which would make the attackers struggling to get the actual pieces of the information from the plaintext. To solve an LWE problem means solving a set of linear equations with errors hidden in the equations. The errors are often tiny.

For example, there is a public matrix A, our secret message or plaintext s, and a random small number to mess the equation e. Normally, without the latter, a cipher is usually defined as $C = As \pmod q$. With LWE however, the equation becomes $C = As + e \pmod q$. Without the small errors, we can solve the system of linear equations with Gaussian Elimination, and thus the encryption algorithm is easier to cracked.

There are two types of LWE :

- Search LWE. This approach is to find s - the secret key from a noisy set of linear equations.

– Decision LWE. This approach is to distinguish an equation with LWE from a pure random noise. Usually, a real LWE set of equations would actually at first look random, but the errors gained from the solutions (gained from lattice reduction algorithms, not Gaussian elimination) of these equations do usually cluster around the actual key.

As the best practice, the size of the randomized error must small enough to make it decipherable for the intended receivers of the encrypted data. Small errors also able to force the attackers to switch into lattice reduction problem (because the errors resulted from attempts to solve these linear equations using Gaussian elimination), which essentially the same as searching shortest vectors (the SVP problem). Which, of course forcing the attackers to convert the system of linear equations into a lattice.

2.2 Lattice

A lattice is defined as a set which contains every single linear combinations of vectors from a basis of $\mathbb{R}^n$. Therefore, we can state that a lattice is theoritically a discrete subset of vector space with the dimension size of n.

Let's say, we have L, which is a lattice, then :

– k is the rank or the dimension size of L
– $\{v_1, v_2, ...v_k\}$ is the basis of L.

L must have vectors of $v_1, v_2, ...v_k$ that each of these vectors are linearly independent and such that $L = \mathbb{Z}$-linear span of $\{v_1, v_2, ...v_k\}$ (Silverman H. Joseph, 2020).

Lattice is one of the key mathematic concepts of the Post-Quantum Cryptography, in fact, of the four PQC algorithms that have been selected by NIST in order to be standardized, three are based on lattice : ML-KEM (based on CRYSTALS-Kyber), ML-DSA (based on CRYSTALS-Dilithium), and FN-DSA (based on Falcon).

2.3 Lattice-based Post-Quantum Cryptography

The lattice-based PQC is a group of cryptographic algorithms that like its name, are based on lattices, and their security is dependent on the hardness on two computational problems that are considered hard even for quantum cryptanalysis algorithms (such as Shor's Algorithm) to solve in the polynomial time : Shortest Vector Problem (SVP) and the Closest Vector Problem (CVP). Example for lattice-based PQC algorithms are :

– CRYSTALS-Kyber. Standardized by NIST into ML-KEM (Module Lattice-based Key Encapsulation Mechanism). This algorithm, just like its pre-standardized version, is a set of algorithms used for two parties in order to establish a shared secret key in a channel considered to be unsafe.
–

- CRYSTALS-Dilithium. Standardized by NIST into ML-DSA (Module Lattice-based Digital Signature Algorithm). It is a set of algorithms that is used to generate as well verify digital signatures.
-
- Falcon (Fast Fourier Lattice-based Compact Signatures Over NTRU). Standardized by NIST into FN-DSA. It's also a DSA like ML-DSA.

Because most lattice-based PQC are using LWE to make their linear equation systems noisier, the problem essentially becomes equivalent (or at least related) to finding unusually short vectors, finding close lattice points, or reducing the lattice basis into a more orthogonal form. This is how the lattice basis reduction plays a vital role - although it doesn't guarantee the lattice-based PQC algorithm to be cracked.

2.4 Lattice Reduction

Lattice reduction is a process to find shortest vectors or closest vectors in lattices (Silverman H. Joseph, 2020). Within a lattice reduction process, the bases considered to be "bad," highly non-orthogonal basis (where the vectors are long and meet at sharp angles) are transformed it into a better version of them, which are reasonably orthogonal (each of their respective vectors are perpendicular to each other, vectors meet at angles closer to 90 degrees, and are relatively short). Why do we need "good" bases? Because good bases make hard lattice problems computationally easier to be solved.

Hadamard's Inequality is a method to tell a basis is a good or bad one.

$$Det(L) \leq \|v1\|.\|v2\|....\|vn\| \tag{1}$$

This inequality clearly states that the determinant of a lattice is always less than or equal to the product of the lengths of its basis vectors. For a bad basis, the product of the vector lengths is drastically larger than the true determinant (the determinant of the lattice where the basis belongs to). For a good basis, the product of the vector lengths are very close to the true determinant of the lattice. A "perfect" basis essentially has the same product of vector lengths with their true determinant. To put it simple, the closer the product of the vector lengths of a basis to the value of its true determinant, the better the basis is.

So, with lattice reduction, we can pick any single bad basis and turn it into a better version of it.

The most known modern lattice reduction method is Lenstra-Lenstra-Lovasz lattice reduction algorithm, often shortened to LLL. Virtually all modern mainstream lattice reduction algorithms are based on the LLL framework. For example, the Block Korkine Zolotarev (BKZ) lattice reduction algorithm uses LLL as an important kind of steps of it.

2.5 Shortest Vector Problem

The SVP (Shortest Vector Problem), like its namesake, is the problem of finding a shortest and non-zero vector in a lattice (Silverman H. Joseph, 2020). For a vector in a lattice to be regarded as the SVP, a vector must be the shortest one (or the shortest ones), out of all vectors in the lattice, and it must be a nonzero vector.

SVP is essential for PQC lattice reduction algorithms because it can help to solve the hidden noises injected in linear equations (the LWE problem) used in PQC algorithms.

2.6 LLL

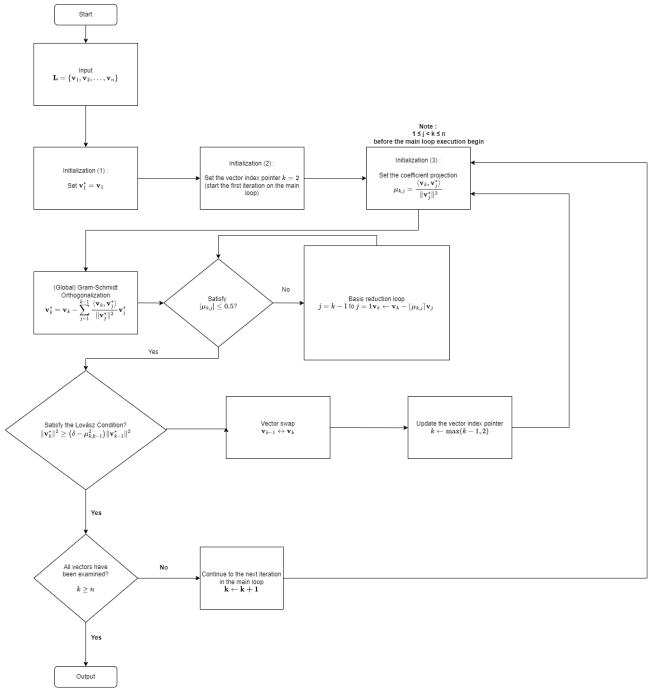

Fig. 1. The LLL algorithm

The LLL (Lenstra-Lenstra-Lovasz) cryptanalysis algorithm, developed by Arjen Lenstra, Hendrik Lenstra, and Laszlo Lovasz in 1982, is a lattice reduction algorithm. Although LLL in present days is largely ineffective to reduce lattices in many PQC algorithms such as ML-KEM and ML-DSA, the LLL is the foundation of modern lattice reduction cryptanalysis. For example, it serves as

the foundation of BKZ, the upscaled version of LLL, by having several LLLs processes executed for a single BKZ process.

In the Fig.1, we can see how the algorithm works. The usual inputs are in the form of matrices, or a set linear equations. Either way, the acceptable form of input in our tool is matrix. However, in the general context, for a single LLL process, the input is in the form of a lattice basis L, which are in the form of a set of basis vectors $\{v_1, v_2, ...v_n\}$. Later for every vector within the set, they will be iteratively processed by Gram-Schmidt orthogonalization, then reduction or swap, and then examined whether the basis at the index fulfills the Lovasz Condition or not. The iteration process, however, is controlled by a pointer variable k which starts from 2nd index to the n-th index (the last vector). In other words, the first vector only acts as a projection reference for Gram-Schmidt orthogonalization (it only acts as the initial orthogonal basis ($\mathbf{v}_1^* = \mathbf{v}_1$)) and size reduction processes of subsequent vectors, but the first vector is never individually subjected to the vector swap mechanism (unless during the first iteration, the second vector fails to meet the Lovász Condition checking) or Lovász Condition checking. After all subsequent vectors have gone through these processes, the loop ends and the algorithm stops.

Unless the initial lattice basis is already a highly orthogonal basis, we will obtain a more orthogonalized and shorter version of the basis. Thus, with a basis that is more orthogonal and shorter, we would have it easier for recovering the LWE small errors hidden in the linear equations that form the public matrix we want to crack.

As an important note, the δ value used in Lovász Condition ranges from 0.25 as the minimum value to 0.99 as the maximum value. A higher δ yields higher quality outputs at the expense of more iterations, and a lower δ leads to fewer iterations at the expense of lower quality outputs. In other words, a higher δ makes the recovery of the LWE errors easier.

2.6.1 Initialization

$$\mathbf{L} = \{v_1, v_2, ...v_n\}$$

We have L, a lattice basis, with n vectors within it. The L contains $\{v_1, v_2, ...v_n\}$. Assume L is a "bad" basis, then our goal is to make it more orthogonal and shorter.

2.6.2 Gram-Schmidt orthogonalization

$$\mathbf{v}_k^* = \mathbf{v}_k - \sum_{j=1}^{k-1} \frac{\langle \mathbf{v}_k, \mathbf{v}_j^* \rangle}{\|\mathbf{v}_j^*\|^2} \mathbf{v}_j^*$$

Right after the initialization, the algorithm processes the basis vectors sequentially. At the global scale (or the outer loop) in L, the algorithm performs Gram-Schmidt orthogonalization (GSO) for all vectors in L, produces the initial orthogonalized $\mathbf{v}_k^*$ and their corresponding projection coefficients $\mu_{k,j}$, with

$1 \leq j < k \leq n$ before the main loop execution begins. Notice that for the first vector ($k = 1$), no projection is needed since it has no preceding vectors. Hence, $\mathbf{v}_1^* = \mathbf{v}_1$.

2.6.3 Main loop After the global Gram-Schmidt orthogonalization process, the index pointer k points at the second vector from the left (the vector of v_2), thus the main loop begins with the first iteration begins at $k = 2$.

2.6.4 Basis reduction loop The algorithm checks the size of the vector, and then performs a size reduction of the vector to ensure that the length of the projection coefficients satisfies :

$$|\mu_{k,j}| \leq 0.5$$

To satisfy the condition, the algorithm uses an internal index pointer, j, whose sole purpose is to become the backward iterator in this basis reduction loop, and this loop starts from $j = k - 1$ down to 1. This ensures that the current target vector $\mathbf{v}_k$ is systematically reduced against all of its preceding vectors in the basis. The vector $\mathbf{v}_k$ is then iteratively updated using the following formula :

$$\mathbf{v}_k \leftarrow \mathbf{v}_k - \lfloor \mu_{k,j} \rceil \mathbf{v}_j$$

Concurrently, to maintain mathematical consistency with the modified vector components, the algorithm must update $\mathbf{v}_k^*$ and the corresponding projection coefficients $\mu_{k,j}$, by fully recomputing the entire Gram-Schmidt orthogonalization sequence to get the updated $\mathbf{v}_k^*$ and $\mu_{k,j}$ values whenever $\mathbf{v}_k$ is modified inside this basis reduction loop.

The basis reduction loop continues until the backward iterator j has fully decreased past 1, signifying that the $\mathbf{v}_k$ has been checked and reduced against every preceding vector in the basis L. Once the basis reduction loop ends, the updated vector is then subjected to the core step of the LLL algorithm - the Lovász Condition checking.

2.6.5 The Lovasz Condition

$$\|\mathbf{v}_k^*\|^2 \geq \left(\delta - \mu_{k,k-1}^2\right) \|\mathbf{v}_{k-1}^*\|^2$$

Using the updated $\mathbf{v}_k^*$ obtained from the preceding basis reduction loop, the algorithm evaluates whether the current vector ordering satisfies the Lovász Condition or not.

Should the basis at the current iteration satisfies the Lovasz Condition, it will proceed to check whether all vectors have been examined. If that's the case ($k >= n$), then the main loop terminates, signifying that the entire basis has been successfully reduced, and followed immediately by the termination of the algorithm altogether.

If not, then the iteration continues and k will point to the next vector ($k \leftarrow k+1$), continuing the main loop until all vectors in the basis L have been fully processed.

However, if the basis fails to satisfy the Lovasz Condition at the current iteration, a vector swap mechanism is executed and followed by a re-evaluation of the basis sequence.

2.6.6 Vector swap If the current basis configuration does not satisfy the Lovász Condition at index k, it implies that the vector $\mathbf{v}_k$ has a significantly shorter orthogonal projection compared to its predecessor, $\mathbf{v}_{k-1}$, which makes the ordering of the vectors within L becomes deficient. To fix this ordering deficiency, a vector swap mechanism must be executed. The order of the two adjacent vectors within the basis is interchanged as follows:

$$\mathbf{v}_{k-1} \leftrightarrow \mathbf{v}_k \tag{2}$$

Since this swap alters the geometric orientation and structural sequence of L, the previously calculated Gram-Schmidt vectors and coefficients become invalid from index $k-1$ onwards. Consequently, the algorithm retreats its main loop iteration counter using the following step :

$$k \leftarrow \max(k-1, 2) \tag{3}$$

This repositioning ensures that the counter shifts backward by one step, forcing the algorithm to recompute the local Gram-Schmidt orthogonalization and re-apply the basis reduction at this lower index before the Lovász Condition can be re-evaluated. As the important note, the max function serves as a logical guardrail to prevent k from accessing indices lower than 2.

2.6.7 Output After all of the vectors have been examined and L meets the Lovasz Condition requirement, the algorithm terminates and returns a more orthogonalized and shortened version of L. This way, with a better lattice basis, we would be able to recover hidden LWE errors embedded within the linear equations which form the public matrix we want to crack.

2.7 BKZ

The BKZ (Block Korkine-Zolotarev) lattice reduction algorithm was first proposed by Clauss-Peter Schnorr in 1987, and in 1994 he and Martin Euchner published the journal of the BKZ algorithm (Ziyu Zhao et al, 2022). This lattice reduction algorithm, while based on LLL, is much more stronger compared to LLL. Nowadays the usage of BKZ in cryptanalysis is common, as the algorithm becoming increasingly important due to the emergence of the lattice-based post-quantum cryptographic algorithms (Jianwei Li et al, 2024).

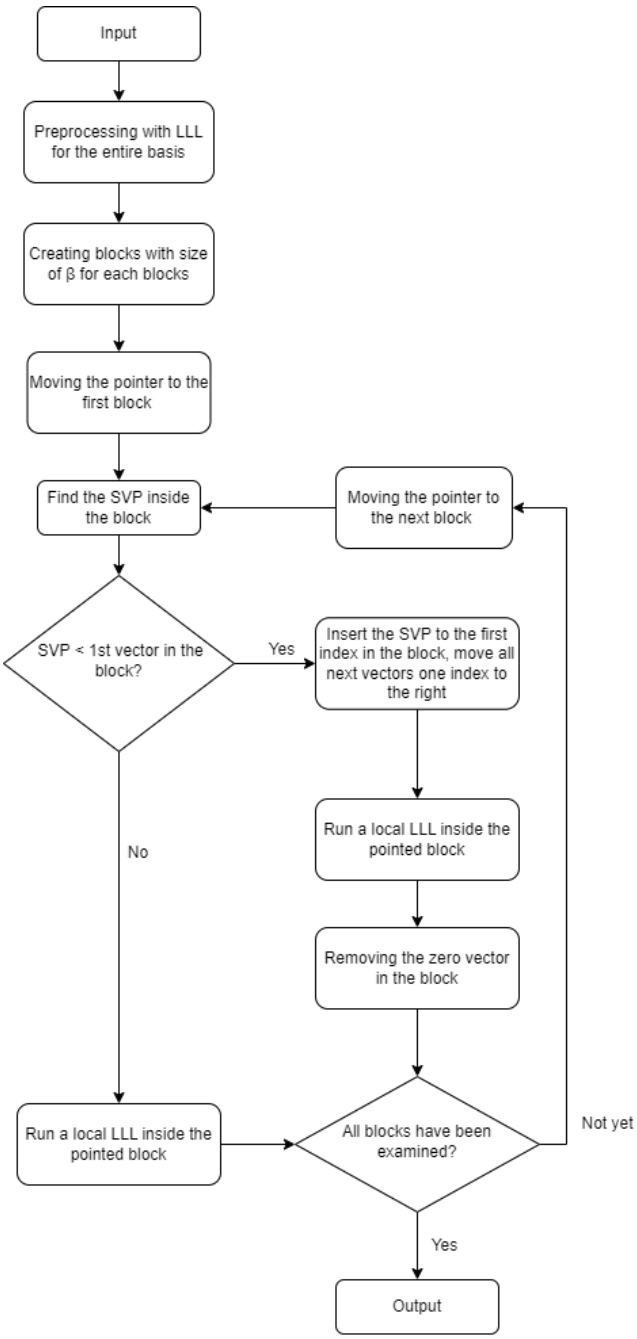

Fig. 2. The BKZ algorithm

2.7.1 Initialization and the first LLL

$$\mathbf{L} = \{v_1, v_2, ...v_n\}$$

We have L, a lattice basis, with n vectors within it. The L contains $\{v_1, v_2, ...v_n\}$. Like in LLL algorithm, our goal is to make L becomes more orthogonal and shorter. The first step here is to run a LLL process to the entire basis vectors within L.

$$\mathbf{LLL}(\mathbf{L}, \delta)$$

The δ-value used in BKZ for all LLL executions within a BKZ algorithm execution is the same, and normally the δ value used is 0.99. This is done to ensure a more orthogonalized and shorter outputs for each LLL executions and the final output of the BKZ execution as well.

2.7.2 The main iteration After the first global LLL process has been completed, the algorithm begins the main iteration. While the LLL process iteratively process the vectors one-by-one across the entire basis (for a single iteration in LLL, only one vector is reduced), BKZ processes vectors within localized regions of the basis called blocks. Think of it as a pointer that points β neighbouring vectors for each iteration, and performs SVP searching, vector swapping, and localized LLL, all exclusively only for all vectors that are currently pointed. After the localized LLL is done, the pointer shifts just one vector to the right.

So in another and simpler words the so-called "block" structure in BKZ behaves more like a sliding window with a fixed block size β that traverses the lattice basis from left to right, and performs SVP enumeration, vector swapping, and LLL only for all vectors within the block.

2.7.3 Block formation For each single iteration, the BKZ algorithm selects a local block from the basis :

$$\mathbf{L}_{[i,h]} = \{v_i, v_{i+1}, ...v_h\}$$

in which

$$\mathbf{h} = \mathbf{max}(i + \beta - 1, n)$$

Here, i is the starting index and h last index of the current block, while β is the fixed block size of the entire BKZ algorithm.

For example, we have $\mathbf{L} = \{v_1, v_2, v_3, v_4, v_5\}$ with $\beta = 3$. The first block would be $\mathbf{L}_{[1,3]} = \{v_1, v_2, v_3\}$ and after the first iteration ends, the next iteration's block would be $\mathbf{L}_{[2,4]} = \{v_2, v_3, v_4\}$. As we see,the "sliding window" only moves by one vector to the left, which is why the BKZ's blocks are overlapping to each other.

As an important note, since

$$\beta$$

-value is the symbolization of the block size, it means that the β-value determines how many neighboring vectors are included inside the current block during each

iteration, and as such, the selection of the β-value has a big role on determining the quality of the BKZ's output. A smaller β produces smaller blocks, which means it produces a faster execution of the algorithm with the cost of the BKZ's output, while a bigger β resulted in slower execution but with better output.

2.7.4 Localized SVP enumeration After the block formation, the BKZ algorithm searches the shortest non-zero vector (SVP) inside the current block. Shall the SVP candidate v is deemed to be shorter enough compared to the currently examined basis vector (as mathematically stated below) :

$$||v|| < \beta||b_i||$$

,

then the SVP candidate v is deemed to be an improvement to the basis structure. Thus, the candidate v is inserted into the basis at the current block position $(v, b_i, b_{i+1}..., b_h)$. Also usually known as nontrivial insertion. Should the condition $||v|| \geq \beta||b_i||$ happens, then no insertion would be done, since the algorithm doesn't consider this case as an impactful improvement of the basis quality.

2.7.5 Localized LLL As the title of this section suggests, within the current block, a LLL process is executed, which only involves all vectors within the current block.

2.7.6 Output Once all blocks within the lattice basis have been examined and processed, and no further nontrivial insertion can be executed, the BKZ algorithm terminates. Like the LLL algorithm, it would produce a shorter and more orthogonalized lattice basis compared to the initial basis. And because the BKZ algorithm is formed from a global LLL, followed by the execution of localized LLL reductions and localized SVP enumerations in each of its blocks, the quality of a BKZ output is generally better (the output is expected to be much shorter and much more orthogonalized) than the output produced from a single LLL process.

Thus, with BKZ, recovering LWE errors hidden in the system of linear equations within a lattice-based post-quantum cryptographic system is much easier to be done with BKZ. The only downside from a BKZ execution compared with LLL, is the computational cost, since a single BKZ process involves several LLL processes.

2.8 lll-attack-runner

lll-attack-runner is a Javascript-based interactive web application for running advanced lattice basis reduction algorithms such as LLL and BKZ. It is developed by Zachary Robert Bennett with the latest update was in January 2026.

Because we aim to create a Javascript-based interactive web application for visualizing lattice-based cryptographic and cryptanalysis algorithms, in which two of them are LLL and BKZ, we will use lll-attack-runner as the backend for our visualization of the LLL and BKZ algorithms. However, since we want to visualize these two algorithms in an intuitive way which is easier to understand and more practical for cryptography learners, we will develop our own frontend for the visualization.

2.9 Herramienta de software para la visualización de la criptografía basada en retículas

Authored by Jaziel D. Flores Rodríguez and his fellow colleagues, this research paper was aimed to create a visualization tool for post-quantum cryptographic algorithms. The main focus was to graphically visualize mathematical concepts behind these post-quantum cryptographic algorithms, particularly lattice concepts which are the basis of these algorithms. The tool which was developed in this research, successfully visualizes these lattice-based post-quantum cryptographic algorithms within dimensions of 2, 3, 4, and 5. However, the LLL process for dimensions higher than 3 is still not implemented in this tool, and the BKZ algorithm is also not implemented in this tool. Which is why we will be implementing the LLL process for dimensions higher than 3 in our visualization tool, as well as implementing the BKZ algorithm which is not implemented in this tool.

3 Architecture

3.1 Overview

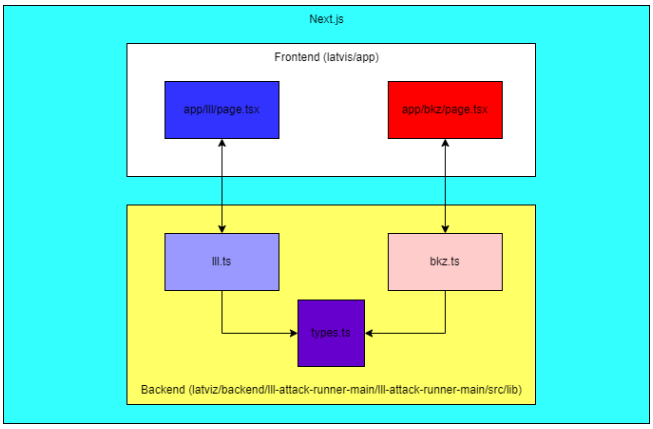

Fig. 3. Program structure of Latviz's LLL and BKZ visualizers

Within our source code repository, there is a segregation between the backend and frontend. The role of these backend source codes is to process the inputs inserted from the user through the interface at the frontend. And obviously the role of the frontend section, aside from getting the inputs from the user, is to do the entire visualization jobs. Because the visualization is our main priority, we can say that the frontend section is much more complex compared to the backend one.

3.2 Backend

We're using Zachary Robert Bennett's lll-attack-runner, which is an interactive and Javascript-based lattice-based cryptanalysis solver, as the backend for our visualization of the LLL and BKZ algorithms. We only use three source codes from lll-attack-runner, which are : lll.ts, bkz.ts, and types.ts. The lll.ts, per its filename, does the LLL algorithm process, and the same goes for bkz.ts. Both source codes needs LLLStep interface from types.ts, and for bkz.ts, it also needs LLLRun from lll.ts.

3.3 Frontend

Within the frontend section, in each of BKZ and LLL algorithm modules, each algorithm has its own folder. Which, in each of these folders, there is a single source code written in Typescript, with naming convention of the algorithm name written in lowercase. Each of these source codes are responsible for running the algorithm process which corresponds to their name. Within each of these source codes, we will use imported functions from Zachary Robert Bennett's lll-attack-runner source codes to run the algorithm process.

We will use Next.js, a React-based framework for building web applications, with the goal to create a user-friendly interface as much as we can for our visualization of the LLL and BKZ algorithms. The user will write their own matrix (or copy-and-paste it from any sources), insert needed values (for example, delta value for both LLL and BKZ algorithm and the block size for BKZ algorithm), and the frontend will pass the values inserted by the user into the backend for the core processing. As what we already explained before in the overview section, the frontend's responsibility is mainly for visualization part.

For the visualization part, we will use D3 for the visualization part, combined with React to create matrices and vectors visualizations. Our secondary objective for the frontend is to create a visualization which have animations for the steps of these algorithms. We believe the animated version of these visualizations can be understand easier by the users.

4 Visualization

4.1 Euclidean Vector Representations for the Visualization of LLL and BKZ

Vector visualization is a very essential as well an intuitive concept to gain proper understanding of linear algebra concepts (Calderone Dan, 2023). This also includes LLL and BKZ algorithms. And so, for both 2D and 3D visualization panels in each modules of LLL and BKZ, we will depict the vectors extracted from the input (the matrix we inserted) as Euclidean vectors within 2D and 3D Cartesian coordinate system.

The reason why we choose Euclidean vectors to represent the processes as well the inputs of the LLL and BKZ is because both algorithms are obviously lattice reduction algorithms, which operate on the vectors within a Euclidean space. As such, visualizing both algorithms through Euclidean vector representations become a natural choice to show the behavior of lattice bases during the lattice reduction processes. Subprocesses within LLL and BKZ such as the Gram-Schmidt Orthogonalization, basis reduction, and vector swapping (LLL), as well the localized LLLs (BKZ) are involving changes of length, direction, and angles of these vectors. To numerically map the Euclidean Space, we will use the Cartesian coordinate system, on the basis that lattice basis vectors are naturally represented numerically as coordinates in Euclidean space. Since the LLL and BKZ algorithms operate through geometric transformations of vectors, we could state that the Cartesian coordinate system is an intuitive way to visually display the changes of the properties of these vectors.

Within the Cartesian coordinate system we will use, whether it is the 2D or the 3D visualization, these vectors are depicted as arrows that are anchored to the origin point of (0,0) in 2D Cartesian coordinate and (0,0,0) in 3D Cartesian coordinate. The reason why we make the (0,0) and the (0,0,0) as the common origin points of all vectors within our visualization panel is to simplify the observation of geometric properties between basis vectors. Thus, by anchoring all vectors to these common origin points, users can directly compare the orientations and magnitudes of vectors without additional coordinate transformations or positional offsets. Conventionally, how the vectors are visualized in linear algebra and vector geometry in the linear algebra (which also includes lattice reduction algorithms like LLL and BKZ) are as arrows originating from the coordinate origin, which in our case, is (0,0) for 2D visualization and (0,0,0) for 3D visualization.

Each arrows has different colouring scheme and is unique in terms of value, as every vectors are unique (as they're linearly independent). No vectors would have more than one arrow that represents them (as they all must be linearly independent).

Thus, by representing basis vectors as Euclidean vectors inside Cartesian coordinate systems which all starts from (0,0) as the "anchor point", and with unique colour for each single vector, we expect the users to be able to visually observe how highly non-orthogonal bases gradually transform into shorter and

more orthogonal bases throughout the LLL and BKZ algorithms in an easier way.

4.2 Representation of Algorithmic Processes in the Visualization System

As what we already stated before, we will build our program based on the LLL and BKZ algorithms. Because our tool is a visualizer, it is obvious that it's necessary to transform the elements from the LLL and BKZ algorithms (such as initial matrix, vector reduction, vector swap, Lovasz status checking, final output, and so) into visual and textual elements. We think that some elements are better visualized, while some others are better shown in textual format.

4.2.1 Lattice Basis Representation of the Input Matrix

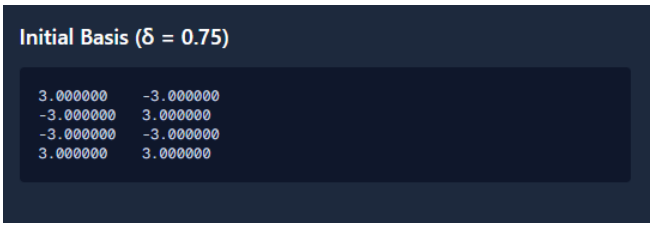

Fig. 4. The initial basis panel (LLL)

The whole visualization process begins after the user insert their matrix in the input field, and then clicked "Insert Matrix" button. Within the frontend system, the inserted matrix is parsed into a 2D array, which represents the basis of the inserted matrix. The outer array represents the entire matrix, while the inner array represents the individual basis vectors which are represented by spatial Cartesian coordinates. These basis vectors will be shown as vectors arranged vertically from top to bottom (just like their matrix counterparts) with the first vector is the topmost vector basis. For the LLL visualization module, the 2D array, alongside with the inserted δ-value will be passed into the runLLL function (which executes a LLL process) and runBKZ function (which executes a BKZ process) in lll.ts and in bkz.ts. For the BKZ visualization module, the 2D array, the δ-value, and the β-value will be passed into the runBKZ function in bkz.ts, which executes a BKZ process.

4.2.2 Euclidean Vectors Representation of the Basis Vectors As what we already stated before, the basis vectors extracted from the input matrix are represented visually as Euclidean vectors inside Cartesian coordinate systems. Thus, each vector is displayed as an arrow originating from the coordinate origin

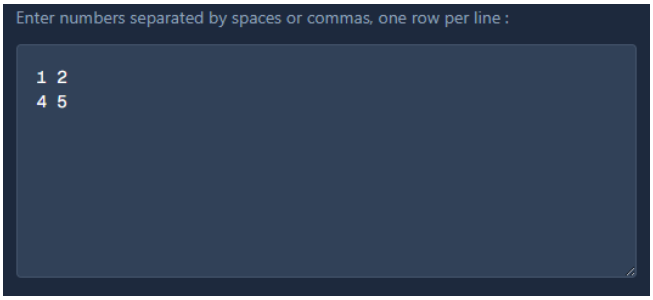

Fig. 5. The input matrix

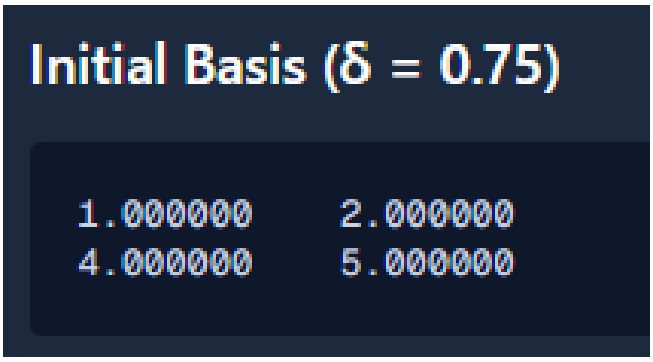

Fig. 6. Basis vectors from the input matrix

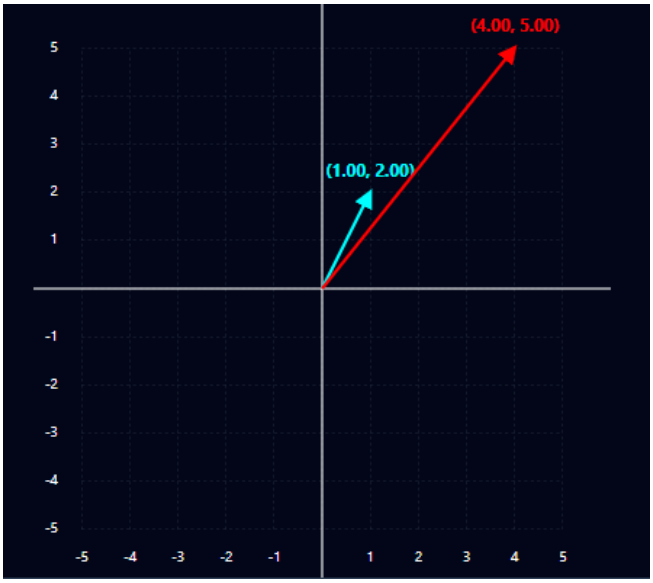

Fig. 7. Basis vectors from the input matrix, represented in the Cartesian coordinate within a Euclidean space

((0,0) in the 2D visualization or (0,0,0) in the 3D visualization), each one having a unique color, and each one pointing toward the coordinates which represent their numerical values - or the elements of the matrix. For example, the vector $v_1 = (4, 5)$ is visualized as an arrow from the origin point of (0,0) toward the point (4,5). Because all vectors are anchored to the origin point, this allows users to observe the length and direction of these vectors, as well the change of them during the reduction process from the beginning until the final output.

4.2.3 Textual Explanation Panel to Represent the Gram-Schmidt Orthogonalization We decide that the Gram-Schmidt Orthogonalization subprocess would be better off explained through textual explanation in a pop-up window when the user clicks "Show Calculations" button in the "Step and Calculation Details" panel (which we will explain it in the section 4.3.3). At first, the LLL module's frontend source code (lll/page.tsx) would call the runLLL function from the backend source code of lll.ts and inserts the needed parameters - which consist of the basis vectors, the δ-value, and the Boolean flag "true". Within the runLLLL function, it would call the function gramSchmidt and the program would insert the basis vectors to the function and the gramSchmidt function would perform a Gram-Schmidt Orthogonalization to these vectors. The function returns the orthogonalized version of these vectors. Later, the function generateGSOCalculations would be called and then these orthogonalized vectors, alongside the original vectors, are inserted to the generateGSOCalculations

function in order to generate a textual display of the mathematical steps behind the Gram-Schmidt Orthogonalization.

4.2.4 Textual Explanation Panel to Represent the Vector Reduction process Like how the Gram-Schmidt Orthogonalization is shown to the users, we will show the vector reduction process in the form of a textual explanation of its calculation steps in a pop-up window. How we technically display the textual explanation of the calculations behind the vector reduction and vector swap processes are roughly similar to the Gram-Schmidt process explanation. When the frontend called runLLL function from the backend source code of lll.ts, after performing Gram-Schmidt Orthonalization and other conditions for the vector reduction to be performed, it would call the function vectorSubtract - which as its name suggests, performs a vector size reduction process. The output produced by vectorSubtract, together with the previous vector and the currently examined vector would then be inserted into a function named generateReduceCalculations in order to generate a textual display of the mathematical steps behind the vector size reduction process.

4.2.5 Textual Explanation Panel to Represent the Vector Swap Process If the current basis doesn't satisfy the Lovasz Condition, it wouldn't only triggered a vector reduction, but also the vector swap after the vector reduction. The currently examined vector would has its index getting switched with the index of the previous vector. Later, the function of generateSwapCalculations would be executed, which generates a textual display of the mathematical steps behind the vector swap process.

4.2.6 Status Panel to Represent the Lovasz Condition Checking The Lovasz Condition is depicted as a status panel of two states ("Satisfied" and "Not Satisfied"). Unlike with the usual mechanism of Lovasz Condition checking, for convenience reasons, we keep display the Lovasz Condition status even when the current process isn't the Lovasz Condition checking as in the common LLL algorithm. This is achieved through the addition of the the function computeLovaszStatus in the app/lll/page.tsx functions to continously checking the Lovasz Condition status in order to keep the Lovasz Status panel has a status at one time, from the first step of the process until the final step. Obviously we have the regular Lovasz Condition checker - the function lovaszCondition in the backend code is to perform Lovasz Condition checking only after the currently examined vector has been reduced. Of course, there is a risk of these two Lovasz Condition checkers would have two different answers at the same time, but we managed to eliminate this risk by employing them into separate responsibilities, as what we already explained before. In the frontend, the runLLL would be executed first, then the produced result (lllResult) contains data of the steps. The steps would then be used by the computeLovaszStatus to visualize the Lovász condition status at each historical step without re-executing the algorithm or affecting the

algorithm's control flow. In a shorter explanation, the computeLovaszStatus only recheck the Lovasz Status based on the existing data gained from LLL execution by the backend.

4.2.7 Text Panel to Represent Shortest Vector Problem The SVP is computed through each of the BKZ iterations. However, the program would only show the SVPs at the final step. It is displayed within a dedicated panel only in the BKZ module.

4.3 Output Visualization and Textual Elements for LLL and BKZ

Visualization of both LLL and BKZ algorithms contains following five visual elements in their output section :

4.3.1 Graph Panel

Fig. 8. The 2D visualization

This is the key visual element. Psychologically speaking, human brain is usually better at understanding visual information compared to textual ones (). We design the graph panel in 2D and 3D projections, both in the fashion of the Cartesian coordinate. Each vector arrows represented in this graph panel have different colour, with their vector written in the same colour as their arrow and placed above the tip of their vector arrow.

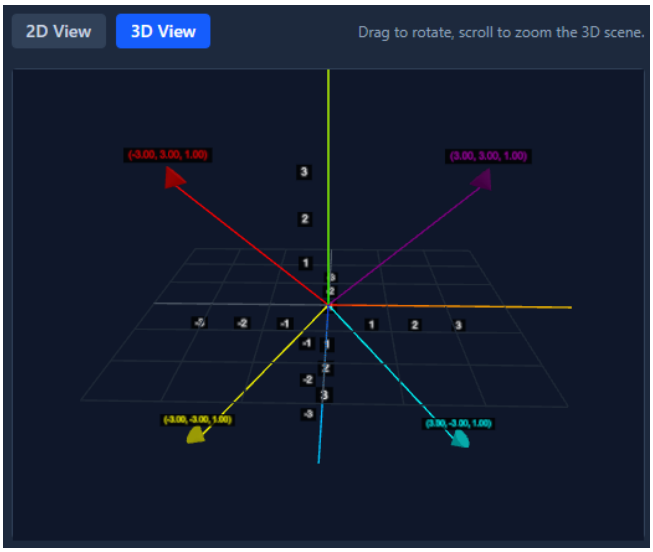

Fig. 9. The 3D visualization

Fig. 10. The Lovasz Condition is displayed in this panel, whether it's satisfied (shown here) or not

4.3.2 Lovasz Status Panel

Only shown in the LLL module, the panel shows the user of the Lovasz status of the current basis. If the current basis fulfill the Lovasz condition, it will show the user a green check mark and "Satisfied" text, and if not, a red cross mark and "Not Satisfied" text will be shown to the user. Unlike in its original counterpart, the Lovasz Condition is continously checked throughout the whole LLL process, not just after the basis reduction step, for convenience reasons.

4.3.3 Step and Calculation Details

Fig. 11. Current step. Click on the "Show Calculations" for the details

This panel will tell the user of the current step (for example, "Basis size reduction"). The user can see the details of the calculation in the step by clicking "Show Calculations" button at the upper right corner of this panel. There are eight types of LLL and BKZ modules :

- "Initial Basis". This step is always the first step to be shown after we insert a matrix and clicked "Process Matrix". Right after we clicked the "Process Matrix" button, the program immediately extracted basis vectors from the input. If the "Step" panels shows that the current step is this step, then it means that the program already extracted basis vectors from the input. This step is one of three type of steps who only be shown once, the other two is "Finished" and "Iteration complete" (BKZ-only).
-
- "Gram-Schmidt orthogonalization". Like its algorithm counterpart, the first "Gram-Schmidt orthogonalization" step would be shown as the second step in a matrix process. In this step, the program performs Gram-Schmidt orthogonalization (GSO) for all vectors from the input, produces the initial orthogonalized $\mathbf{v}_k^*$ and their corresponding projection coefficients $\mu_{k,j}$, with $1 \leq j < k \leq n$. This step also would be executed should a basis doesn't satisfies the Lovasz Condition (in this program, depicted in the form of a panel on its own).
-
- "Basis reduction". It represents the basis reduction step from the original algorithm, in which a vector that is currently examined (started from the second vector in the basis from the input), having been orthogonalized globally during the Gram-Schmidt orthogonalization step previously together with all other vectors in the basis from the input, is being

subtracted by the previous vector (), in order to shorten the currently examined vector. This step also would be executed should a basis doesn't satisfies the Lovasz Condition.

–
– "Basis swap". This step is only executed and shown if during the "real" Lovasz Condition checking (which is executed after the basis reduction step, just like in the original algorithm), the Lovasz Condition isn't satisfied. In this step, like its namesake, the vector which are currently examined, has its index swapped with the previous vector.
–
– "Start LLL Process #n". Only in the BKZ module, when this step is shown, it means that the LLL process in the current block is being executed (in the n-th iteration).
–
– "Complete LLL Process #n". Also only in the BKZ module, when this step is shown, it means that the LLL process in the current block is has been completed (in the n-th iteration).
–
– "Iteration Complete". Like two previous steps, this is also a BKZ module-only type of step. Like its namesake, if this step is shown, then an iteration is stated to be completed, and unless it's the final iteration, the next iteration begins immediately.
–
– "Finished". This step is in both LLL and BKZ modules. If this state is shown, then the program (LLL or BKZ, depends on what the module we currently use) terminates. In the context of their respective algorithms, if this step is reached, for LLL, all vectors has been examined and the Lovasz Condition is satisfied for the basis from the input, which in the other words, the basis has been successfully shortened and orthogonalized in accordance to the δ-value. For BKZ, all blocks has been examined (the Lovasz Condition is checked inside each block as part of the local LLL process).

4.3.4 Current Basis Panel Like its namesake, this panel's sole function is

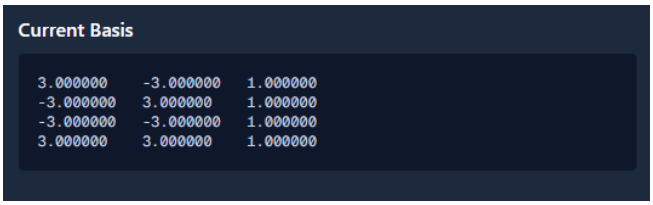

Fig. 12. The current basis as with the current step

to show the user the current basis.

4.3.5 Shortest Vector Panel

Fig. 13. The SVP we gained, which only shown in the final step of the BKZ process

This panel's sole function is to inform the user the shortest vectors found in the current basis. Only shown in the last step.

4.4 Visualization of the LLL Algorithm

First, we have the LLL algorithm. The input panel for this algorithm has matrix and delta value insertion fields, and located in the left half of the page. After the user inserted matrix and delta value properly, and clicked "Process Matrix" button, user can see the "Initial Matrix" as well "Initial Basis" panels below the input panel. To the right half of the page, there is the output section, contains four of five visual and textual information panels, without Shortest Vector Panel.

User can click "Back" and "Next" button in the upper part of the output section to see the previous and next steps of the LLL algorithm process. All visual and textual information panels in the output section are subjects of change according to the current step of the LLL algorithm process.

[insert the pipeline algorithm from input to blabla diagram here lol]

4.5 Visualization of the BKZ Algorithm

Next, we have the BKZ algorithm, which is one of the more powerful lattice reduction algorithm. Because theoritically this algorithm could be comprised of multiple LLL processes, we design the output section to show the user the number of LLL processes as well an additional information called "Current Process", which provides the user of the steps that is unique to BKZ algorithm, which are the current number of the LLL process and SVP enumeration.

The only differences with LLL page is in the input panel, which has an additional, mandatory field for the block size, and in the output section, in which has Shortest Vector panel. Moreover, in the upper part of the output section, we place four additional information, which are "Block Size", "Delta", "Total LLL Processes", and "Current Process". Out of all of these additional information, only "Current Process" isn't static.

Aside from additional panels and input field, the rest of the visualization and basically mechanism of the output's UI is the same as the LLL ones.

[insert the pipeline algorithm from input to blabla diagram here lol]

5 Result

5.1 Surveying Method

insert how we get the data, how we design questions, what are questions, etc

5.2 LLL Visualization

insert the result diagrams of the survey questions related with LLL visualizer

5.3 BKZ Visualization

insert the result diagrams of the survey questions related with BKZ visualizer

6 Evaluation

7 Conclusion

References

1. Ziyu Zhao, Jintai Ding. (2022). Practical Improvements on BKZ Algorithm. National Institute of Standards and Technology `https://csrc.nist.gov/csrc/media/Events/2022/fourth-pqc-standardization-conference/documents/papers/practical-improvements-on-bkz-algorithm-pqc2022.pdf`
2. Mayer, RE. (2001). Multimedia Learning. Cambridge University Press.
3. Christopher D. Hundhausen, Sarah A. Douglas, John Stasko. (2002). A Meta-Study of Algorithm Visualization Effectiveness. Journal of Visual Languages and Computing, 13(3), 259-290.
4. A.Cihan Konyaliouglu, Ahmet Isik, Abdullah Kaplan, Seyfullah Hizarci, Merve Durkaya. (2011). Visualization approach in teaching process of linear algebra. Procedia Social and Behavioral Sciences 15 (2011) 4040-4044.
5. Wang Yunfei, Jin Xin, Liu Junyu. (2025). Learning with errors may remain hard against quantum holographic attacks arXiv:2509.22833
6. Silverman, Joseph H. (2020). An Introduction to Lattices, Lattice Reduction, and Lattice-Based Cryptography. IAS/Park City Mathematics Series Graduate Summer School Lecture Notes.
7. Meyer Alexander. (2024). Post-Quantum Cryptography: An Analysis of Code-Based and Lattice-Based Cryptosystem. arXiv:2505.08791.
8. Calderone, Dan. (2023). Vector Visualizations. Github.
9. Jianwei Li, Phong Q Nguyen. A Complete Analysis of the BKZ Lattice Reduction Algorithm. Journal of Cryptology, In press, ⟨10.1007/s00145-024-09527-0⟩. ⟨hal-04728596⟩
10. LNCS Homepage, `http://www.springer.com/lncs`, last accessed 2023/10/25

$$x + y = z \tag{4}$$

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead